Computational Economics

Lecture: Simulation-Based Learning

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **From expected updates to sampled targets**
  - What an update aims at, and how simulation supplies its inputs
- **Two targets: Monte Carlo and TD(0)**
  - Complete returns versus bootstrapped targets: derivations and worked updates
- **Comparing targets and learning reliably**
  - Practical lessons on step sizes, visits, and accuracy
- **Optional extension**
  - Multi-step returns and TD(λ)

From the Bellman Equation to Sampled Targets

$$V^{\sigma}(s) = \underbrace{r(s, \sigma(s)) + \beta \, \mathbb{E}[V^{\sigma}(s') \mid s, a = \sigma(s)]}_{\text{Bellman target}} .$$

average over all possible next states

Replacing V^{σ} on the right by the current V gives the **update target** $T_{\sigma} V$.

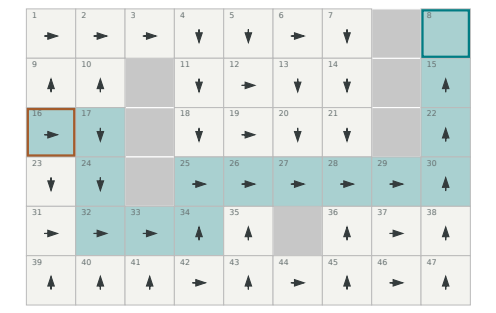
- **Continuous-state example:** For continuous future productivity, integrate over its conditional distribution.
- Simulation replaces costly integration with sampled transitions and rewards.

Can realized experience supply an update target?

We will follow one maze episode through two methods: **Monte Carlo (MC)** and **temporal-difference (TD)** learning.

First observe the experience; then decide what target to construct from it.

One Maze Episode, Two Ways to Learn



Arrows: policy actions. Orange outline: usual start; teal outline: goal. Teal cells lie on the route from cell 16.

Random delays: An action succeeds with probability $.9$; otherwise the agent stays in place.

Observed episode: Start at cell 22 and take up at every step.

Time	Transition	Reward
0	$22 \to 22$ (delay)	-1
1	$22 \to 15$	-1
2	$15 \to 8$ (terminal)	-1

Comparison convention: Both methods use the same policy and estimate the same V^{σ}.

A **trajectory** is a sequence of states, actions, and rewards. An **episode** is a complete trajectory from a starting state to termination (here, the goal).

MC: Wait for the Complete Return

The **complete discounted return**, for terminal time T, is

$$G_t = \sum_{j=0}^{T-t-1} \beta^j r_{t+j+1}, \qquad V^\sigma(s) = \mathbb{E}_\sigma[G_t \mid s_t = s].$$

The same maze episode

$$s_0: \text{ cell } 22 \xrightarrow{r_1=-1} s_1: \text{ cell } 22 \xrightarrow{r_2=-1} s_2: \text{ cell } 15 \xrightarrow{r_3=-1} s_3 = s_T: \text{ cell } 8 \quad (T=3).$$

After termination, work backward from the final reward:

$$\begin{aligned} G_2 &= r_3 = -1, \\ G_1 &= r_2 + \beta G_2 = -1 + .9(-1) = -1.9, \\ G_0 &= r_1 + \beta G_1 = -1 + .9(-1.9) = -2.71. \end{aligned}$$

First visit means the first time a state appears within an episode. Cell 22 appears at $t = 0$ and $t = 1$: use $G_0 = -2.71$ once. For cell 15, use $G_2 = -1$.

From Sample Returns to an Incremental Estimate

For n independent first-visit returns at s, split off the last return:

$$V_n(s) = \frac{1}{n}\sum_{j=1}^{n} G^{[j]}(s) = \frac{n-1}{n} V_{n-1}(s) + \frac{1}{n} G^{[n]}(s).$$

Rearranging gives the recursive update:

$$V_n(s) = V_{n-1}(s) + \frac{1}{n}\left[G^{[n]}(s) - V_{n-1}(s)\right].$$

With $V_1(s) = G^{[1]}(s)$, induction verifies the sample-mean formula for every n.

Two episodes starting at cell 22

- Previous trajectory: $22 \to 22 \to 15 \to 8$. Return -2.71; first estimate $V_1(22) = -2.71$.
- Next trajectory: $22 \to 15 \to 8$. Return -1.9; new mean $V_2(22) = -2.305$.

- Complete returns are unbiased targets; averaging reduces sampling error.
- Each target becomes available after its episode terminates.

First-Visit MC: Policy Evaluation Algorithm

0. **Inputs.** Fixed policy σ, discount β, starting-state distribution μ_0, and episode budget K. Episodes terminate with finite expected length.
1. **Initialize.** Set $V(s) = 0$ and $N(s) = 0$ for every state; keep terminal values zero.
2. **Generate an episode.** Draw $s_0 \sim \mu_0$, follow σ, and record states and rewards until termination.
3. **Compute returns backward.** Set $G_T = 0$, then

$$G_t = r_{t+1} + \beta G_{t+1}, \qquad t = T-1, \ldots, 0.$$

4. **Update each first-visited state once.** At its earliest occurrence t, set

$$N(s_t) \leftarrow N(s_t) + 1, \qquad V(s_t) \leftarrow V(s_t) + \frac{G_t - V(s_t)}{N(s_t)}.$$

5. **Repeat and return.** Repeat Steps 2–4 for K episodes. Return V and the visit counts N.

TD: Update Before the Episode Ends

MC waits for the complete return. TD replaces the unobserved remaining return with the current continuation estimate:

$$\underbrace{G_t = r_{t+1} + \beta G_{t+1}}_{\text{MC: wait for future rewards}} \quad \longrightarrow \quad \underbrace{r_{t+1} + \beta V_t(s_{t+1})}_{\text{TD target: available after one transition}} .$$

Bootstrapping means using an estimated continuation value in the target.

The first transition of the same episode

$$s_0 : \text{ cell } 22 \xrightarrow{r_1 = -1} s_1 : \text{ cell } 22 \qquad \text{future rewards not yet observed}$$

$$r_1 + \beta V_0(s_1) = -1 + .9 V_0(22).$$

Averaging this target over possible next transitions recovers the expected DP update:

$$\mathbb{E}[r_{t+1} + \beta V_t(s_{t+1}) \mid s_t = s, V_t] = (T_\sigma V_t)(s).$$

TD(0): Move Toward the TD Target

$$\delta_t = \underbrace{r_{t+1} + \beta V_t(s_{t+1})}_{\text{TD target}} - V_t(s_t), \qquad V_{t+1}(s_t) = V_t(s_t) + \alpha_n(s_t)\delta_t.$$

The **TD error** is target minus estimate; n counts updates at s_t.

The same episode; initialize $V_0 = 0$, use $\alpha_n(s) = .5$

$$s_0 : \text{ cell } 22 \xrightarrow{r_1=-1} s_1 : \text{ cell } 22 \xrightarrow{r_2=-1} s_2 : \text{ cell } 15 \xrightarrow{r_3=-1} s_3 = s_T : \text{ cell } 8.$$

t	TD target: symbols, then numbers	Updated value
0	$r_1 + \beta V_0(s_1) = -1 + .9(0) = -1$	$V_1(22) = 0 + .5(-1) = -.5$
1	$r_2 + \beta V_1(s_2) = -1 + .9(0) = -1$	$V_2(22) = -.5 + .5(-.5) = -.75$
2	$r_3 + \beta V_2(s_3) = -1 + .9(0) = -1$	$V_3(15) = 0 + .5(-1) = -.5$

Only the current state is updated; all other estimates retain their previous values.

TD(0) Policy Evaluation: Algorithm

0. **Inputs.** Fixed policy σ, discount β, starting distribution μ_0, step budget B, and state-visit step sizes $\alpha_n(s)$.
1. **Initialize.** Set $V = 0$, $N = 0$, and draw $s \sim \mu_0$. Keep terminal values zero.
2. **Observe one transition.** Take $a = \sigma(s)$, receive $r = r(s, a)$, and observe s'.
3. **Update one state.** Increment $N(s)$, set $n = N(s)$, and compute the TD target before changing $V(s)$:
$$V(s) \leftarrow V(s) + \alpha_n(s)\,[r + \beta V(s') - V(s)].$$
4. **Advance or restart.** Set $s = s'$. If terminal, start a new episode from μ_0.
5. **Repeat and return.** Repeat Steps 2–4 for B transitions. Return V and N.

Different Targets, the Same Objective

Objective: Estimate the value of each state under the same policy.

Method	What the target uses	When it is available
DP	An expectation calculated from the model	After model calculation
MC	A complete realized return	After the episode
TD	One reward plus estimated continuation	After one transition

What makes sampled updates useful? How much each update changes the estimate, which states receive updates, and how accuracy is checked.

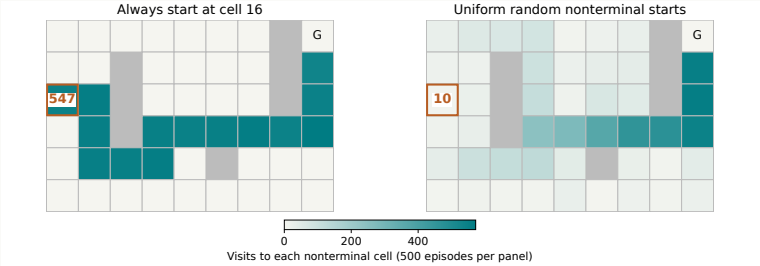
TD Step Size Controls the Size of Each Update



Same policy and observed transitions; both TD runs start from zero.

- Large TD steps react quickly to new transitions but fluctuate more.
- Small TD steps change estimates more slowly and smooth the fluctuations.

Starting States Determine Which Values Can Be Learned



Colors: Darker teal = more visits; white = no visits; gray = wall; G = goal.

Orange cell 16: 547 visits with fixed starts; 10 with random starts, over 500 episodes in each panel. Repeat visits during delays count again, so visits can exceed episodes.

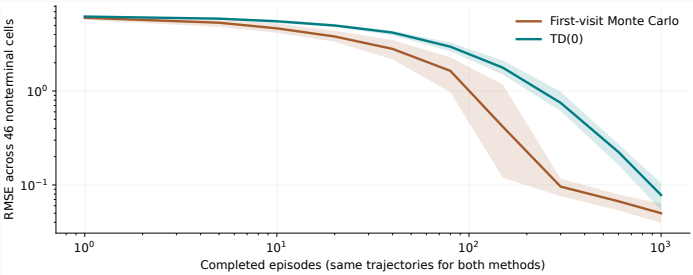
- Fixed starts leave 32 nonterminal cells without updates.
- Random starts reach all 46 nonterminal cells, so we use them in the comparison.

Measuring Accuracy against Exact DP

For this small maze, exact DP gives a benchmark.

1. **Compute the reference values.** Solve the fixed-policy Bellman equations using the known model.
2. **Give MC and TD the same experience.** Use the same policy and episodes, with random starting cells. MC averages returns; TD uses decreasing step sizes.
3. **Compare estimates with the reference.** Summarize their differences across all 46 nonterminal cells using root mean squared error (RMSE). Smaller means closer to the exact values.

MC and TD Accuracy in the Maze



Horizontal: completed episodes (experience). **Vertical:** error against exact DP (lower is better). Lines average 20 runs; bands show variation across runs.

- Both methods improve with experience; MC has lower error here.
- MC gets complete returns throughout the maze; TD must learn continuation values. Starting states and step sizes also affect the comparison.
- Chapter 6's random walk favors TD: this result is specific to the task and learning settings.

Optional: Between TD and Monte Carlo

We can wait for more rewards before using a continuation estimate.

- **After one transition:** the TD(0) target

$$r_{t+1} + \beta V_t(s_{t+1}).$$

- **After two transitions:** use two rewards and the current estimate

$$r_{t+1} + \beta r_{t+2} + \beta^2 V_{t+1}(s_{t+2}).$$

- **At termination:** use the complete return G_t, with no continuation estimate.

If the episode ends earlier, stop at termination and use the remaining realized rewards.

TD(λ) combines targets with different waiting lengths. It connects one-step TD with learning from longer returns.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 5: Monte Carlo Methods
   - Chapter 6: Temporal-Difference Learning
   - Chapter 7: n-step Bootstrapping (optional)
   - Chapter 12: Eligibility Traces (optional)